CS486 FINAL EXAM
Symbolic Problem-Solving Cheatsheet

Built around the official 10-point FINAL REVIEW structure and the five mock finals.

Notation and terminology aligned with *Fundamentals of Database Systems*, 7th ed. (Elmasri & Navathe), especially Chapters 3, 5–9, 14–22.

Purpose. This is not a prose summary. It is a symbol-heavy exam manual: operator notation, formulas, decision rules, reusable derivations, and answer templates for ER modeling, RA/TRC/SQL, constraints, serializability, 2PL/deadlocks, recovery, FDs/normalization, concurrency anomalies, and query-cost estimation.

Contents

0. Master Symbol Sheet

0.1 Logic and Set Symbols

Symbol	Meaning	Symbol	Meaning
$\in$	belongs to	$\notin$	does not belong to
$\subseteq$	subset	$\cup$	union
$\cap$	intersection	$-$ or $\setminus$	set difference
$\times$	Cartesian product	$\lvert R\rvert$	cardinality (number of tuples)
$\exists$	there exists	$\forall$	for every
$\neg$	NOT	$\wedge$	AND
$\vee$	OR	$\Rightarrow$	implies
$\Leftrightarrow$	iff	$\emptyset$	empty set

0.2 Core Relational Algebra Symbols

Symbol	Name	General form / meaning
σ	SELECT	$\sigma_\theta(R)$: keep tuples of R satisfying condition θ
π	PROJECT	$\pi_{A_1,\ldots,A_k}(R)$: keep named attributes; formal RA removes duplicates
ρ	RENAME	$\rho_{S(B_1,\ldots,B_n)}(R)$ or $\rho_S(R)$
$\cup$	UNION	$R \cup S$, requires union compatibility
$\cap$	INTERSECTION	$R \cap S$, requires union compatibility
$-$	DIFFERENCE	$R - S$, requires union compatibility
$\times$	CARTESIAN PRODUCT	$R \times S$
$\bowtie$	NATURAL JOIN	$R \bowtie S$
$\bowtie_\theta$	THETA JOIN	$R \bowtie_\theta S = \sigma_\theta(R \times S)$
$\div$	DIVISION	$R(Z) \div S(X)$, where $X \subseteq Z$; answers "for all" queries
$\mathcal{F}$	AGGREGATE*	Extended RA: e.g. $Dno\,\mathcal{F}_{\mathrm{COUNT}(*)}(EMPLOYEE)$

*Aggregate/grouping notation varies by textbook/lecturer. Use the notation taught in class if it differs.

0.3 Transaction / Concurrency Symbols

$r_i(X)$	T_i reads item X	$w_i(X)$	T_i writes item X
c_i	T_i commits	a_i	T_i aborts
$S_i(X)$	shared/read lock on X	$X_i(X)$	exclusive/write lock on X
$U_i(X)$	unlock X	$T_i \to T_j$	precedence/wait-for edge

0.4 Functional Dependency Symbols

$X \to Y$	functional dependency: equal X values force equal Y values
X^+	closure of attribute set X under FD set F
F^+	all FDs logically implied by F
$X \twoheadrightarrow Y$	multivalued dependency (usually beyond this final unless explicitly asked)
K	candidate key / superkey depending context

1. ER Modeling and ER-to-Relational Mapping (1 point)

1.1 What to draw

For every entity set, show: entity name, attributes, key attributes, weak/partial keys if any, relationship diamonds/lines, cardinality ratios (1:1, 1:N, M:N), and total/partial participation when specified.

Language decoder.

- "identified by" / "unique" ⇒ candidate key.
- "unique only within X" ⇒ scoped key, normally owner key + partial key.
- "each A belongs to exactly one B" ⇒ $A : N \to B : 1$ with total participation of A.
- "A may have many B and B may have many A" ⇒ M:N.
- "at most one" ⇒ maximum cardinality 1; "one or more" ⇒ minimum cardinality 1.

1.2 Weak / scoped identification

If TeamName is unique only inside League:

$$\text{Team}(\underline{LeagueName}, \underline{TeamName}, \ldots)$$

If JerseyNo is unique only inside Team:

$$\text{Player}(\underline{LeagueName}, \underline{TeamName}, \underline{JerseyNo}, \ldots)$$

The full owner key propagates into the weak/scoped entity key.

1.3 Mapping formulas

$$\begin{aligned}
\text{Strong entity } E &: E(\underline{K}, A_1, \ldots, A_n) \\
1{:}N &: \text{put key of 1-side as FK in N-side} \\
M{:}N &: R(\underline{K_A, K_B}, \text{relationship attributes}) \\
1{:}1 &: \text{FK on one side, preferably total-participation side; often UNIQUE} \\
\text{Weak entity } W &: W(\underline{K_{owner}, PartialKey}, \ldots)
\end{aligned}$$

Trap: If the problem intentionally states local uniqueness, do not invent a global surrogate key and ignore the composite identification logic.

2. Relational Algebra, TRC, and SQL (2 points)

2.1 SELECT σ and PROJECT π

Textbook-style general forms:

$$\boxed{\sigma_{\langle condition \rangle}(R)} \qquad \boxed{\pi_{A_1,\ldots,A_k}(R)}$$

Example:

$$\pi_{Name}\left(\sigma_{Faculty='CS'}(Student)\right)$$

Boolean conditions:

$$\theta ::= A \; op \; c \mid A \; op \; B \mid \theta_1 \wedge \theta_2 \mid \theta_1 \vee \theta_2 \mid \neg\theta$$

where $op \in \{=, \neq, <, \leq, >, \geq\}$.

2.2 Rename ρ

Needed for self-joins and attribute-name alignment:

$$\rho_{P_1(m_1,mo_1,t_1)}(Product), \qquad \rho_{P_2(m_2,mo_2,t_2)}(Product)$$

2.3 Join notation

$$R \bowtie_{R.A=S.B} S \equiv \sigma_{R.A=S.B}(R \times S)$$

Natural join:

$$R \bowtie S$$

when common attribute names are intended as equality conditions.

2.4 "At least two distinct" using self-join

Given $L(maker, model)$ for laptops:

$$L = \pi_{maker,model}\left(\sigma_{type='laptop'}(Product)\right)$$

Rename twice:

$$L_1 = \rho_{L_1(m_1,mo_1)}(L), \quad L_2 = \rho_{L_2(m_2,mo_2)}(L)$$

Then:

$$AtLeast2 = \pi_{m_1}\left(\sigma_{m_1=m_2 \wedge mo_1 \neq mo_2}(L_1 \times L_2)\right)$$

2.5 "Exactly two" without COUNT in classical RA

Build $AtLeast2$ and $AtLeast3$:

$$\boxed{Exactly2 = AtLeast2 - AtLeast3}$$

For $AtLeast3$, use three renamed copies with pairwise distinct models:

$$mo_1 \neq mo_2 \wedge mo_1 \neq mo_3 \wedge mo_2 \neq mo_3$$

2.6 DIVISION ÷ for "all/every"

The book uses division for universal-quantification style queries. If:

$$R(Z) \div S(X), \qquad X \subseteq Z, \quad Y = Z - X$$

then the result is $T(Y)$ containing those Y-values that appear in R paired with *every* tuple in S.
Equivalent construction using only $\pi, \times, -$:

$$\begin{aligned} T_1 &\leftarrow \pi_Y(R) \\ T_2 &\leftarrow \pi_Y\big((S \times T_1) - R\big) \\ T &\leftarrow T_1 - T_2 \end{aligned}$$

Mental translation: "find y related to all required x" ⇒ either division $R(y, x) \div S(x)$ or double negation: there is no required x for which the relationship is missing.

2.7 Tuple Relational Calculus (TRC)

General form:

$$\boxed{\{\, t \mid P(t) \,\}}$$

Tuple variables range over tuples. Typical atoms:

$$R(t), \quad t[A] = u[B], \quad t[A] > 500$$

Quantifiers/connectives:

$$\exists t\ P(t), \qquad \forall t\ P(t), \qquad \neg P, \quad P \wedge Q, \quad P \vee Q, \quad P \Rightarrow Q$$

At least one

$$\{s \mid Student(s) \wedge \exists e \exists c(Enroll(e) \wedge Course(c) \wedge e.SID = s.SID \wedge e.CID = c.CID)\}$$

Never / none

$$\{p \mid Patient(p) \wedge \neg\exists a(Appointment(a) \wedge a.PID = p.PID \wedge a.Fee > 500)\}$$

Every / all: direct implication form

$$\forall r\,(Required(r) \Rightarrow Done(s, r))$$

Safer exam form (counterexample form):

$$\boxed{\neg\exists r\,(Required(r) \wedge \neg Done(s, r))}$$

2.8 SQL quantifier patterns

NOT EXISTS for "never"

```
SELECT ...
FROM Candidate c
WHERE NOT EXISTS (
  SELECT 1
  FROM BadThing b
  WHERE b.cid = c.id
);
```

Double NOT EXISTS for "every"

```
SELECT ...
FROM Candidate c
WHERE NOT EXISTS (
  SELECT 1
  FROM Required r
  WHERE NOT EXISTS (
    SELECT 1
    FROM Done d
    WHERE d.cid = c.id
      AND d.item = r.item
  )
);
```

Top per group, preserve ties

```
WITH g AS (
  SELECT group_col, item_col, COUNT(*) AS cnt
  FROM T
  GROUP BY group_col, item_col
), r AS (
  SELECT g.*,
         DENSE_RANK() OVER
         (PARTITION BY group_col ORDER BY cnt DESC) AS rk
  FROM g
)
SELECT * FROM r WHERE rk = 1;
```

NULL trap: Prefer `NOT EXISTS` to `NOT IN (subquery)` when the subquery can produce NULL. Three-valued logic can make `NOT IN` unexpectedly UNKNOWN.

3. Database Constraints (1 point)

3.1 Formal constraint pattern

Most business rules can be written as "no violating tuple combination exists":

$$\boxed{\neg\exists t_1 \cdots \exists t_k\; Violation(t_1, \ldots, t_k)}$$

Example: employee and assigned project must belong to same department:

$$\neg\exists e \exists p \exists w \big(Employee(e) \wedge Project(p) \wedge WorksOn(w) \wedge w.EID = e.EID \wedge w.PID = p.PID \wedge e.Dept \neq p.Dept\big)$$

3.2 SQL mechanism decision table

Rule type	Mechanism	Example
Domain / row-local	`CHECK`	$0 < Hours \leq 40$
Entity identity	`PRIMARY KEY`, `UNIQUE`	no duplicate key
Referential	`FOREIGN KEY`	child must reference parent
Cross-table	ASSERTION (conceptually) / trigger	Employee.Dept = Project.Dept
Temporal/cross-row	trigger/procedure	no overlapping booking; status valid at order date

3.3 Assertion-style expression

Conceptual SQL standard form:

```
CREATE ASSERTION rule_name CHECK (
  NOT EXISTS (
    SELECT 1
    FROM ...
    WHERE violation_condition
  )
);
```

Many production DBMSs do not implement general `CREATE ASSERTION`; use a trigger if required.

4. Serialization, Recoverability, 2PL, and Deadlocks (2 points)

4.1 Conflict relation

Two operations from different transactions conflict iff:

$$\boxed{\text{same item } \wedge \text{ different transactions } \wedge \text{ at least one write}}$$

	$r_j(X)$	$w_j(X)$	different item Y
$r_i(X)$	no	yes	no
$w_i(X)$	yes	yes	no

If operation of T_i precedes a conflicting operation of T_j, add:

$$\boxed{T_i \to T_j}$$

4.2 Precedence graph theorem

$$\boxed{S \text{ is conflict-serializable } \iff P(S) \text{ is acyclic}}$$

If acyclic, every topological ordering of $P(S)$ is a conflict-equivalent serial schedule.

4.3 Example edge extraction

For:

$$w_1(X) \quad r_2(X) \quad w_2(Y) \quad r_3(Y) \quad w_3(X)$$

conflicts give:

$$T_1 \to T_2 \quad (X), \qquad T_2 \to T_3 \quad (Y), \qquad T_1 \to T_3 \quad (X)$$

No cycle $\Rightarrow$ serial order T_1, T_2, T_3.

4.4 Recoverable, cascadeless, strict

Suppose T_j reads X from T_i (notation: $w_i(X) \prec r_j(X)$ and no intervening write to X).

$$\begin{aligned} \text{Recoverable: } & r_j(X) \text{ reads from } w_i(X) \Rightarrow c_i \prec c_j \\ \text{Cascadeless: } & r_j(X) \text{ reads from } w_i(X) \Rightarrow c_i \prec r_j(X) \\ \text{Strict: } & w_i(X) \prec o_j(X)\ (o_j \in \{r_j, w_j\}) \Rightarrow c_i/a_i \prec o_j(X) \end{aligned}$$

Hierarchy:

$$\boxed{\textit{Strict} \Rightarrow \textit{Cascadeless} \Rightarrow \textit{Recoverable}}$$

4.5 Two-Phase Locking (2PL)

Basic 2PL has two phases:

$$\underbrace{\text{acquire locks}}_{\text{growing}} \;\Big|\; \underbrace{\text{release locks}}_{\text{shrinking}}$$

After the first unlock, a transaction may not acquire another lock.
Strict 2PL: hold all exclusive/write locks until commit/abort. A common strict sequence:

$$X_i(X)\ w_i(X)\ \cdots\ c_i\ U_i(X)$$

4.6 Lock compatibility

requested \ held	S	X
S	compatible	incompatible
X	incompatible	incompatible

4.7 Wait-for graph and deadlock

Edge:

$$T_i \to T_j \quad \text{iff } T_i \text{ waits for a lock held by } T_j$$

Deadlock criterion:

$$\boxed{\text{wait-for graph contains a directed cycle}}$$

Classic:

$$T_1:\ X_1(A),\ \textit{request}\ X_1(B) \qquad T_2:\ X_2(B),\ \textit{request}\ X_2(A)$$

Hence $T_1 \to T_2$ and $T_2 \to T_1$.

4.8 Wait-die vs wound-wait

Let $TS(T_{old}) < TS(T_{young})$.

	older requests younger's lock	younger requests older's lock
Wait-die	older waits	younger aborts (dies)
Wound-wait	younger is aborted (wounded)	younger waits

5. Recovery (1 point)

5.1 Log record notation

Common forms:

$$\langle START\ T_i\rangle, \quad \langle T_i, X, old, new\rangle, \quad \langle COMMIT\ T_i\rangle, \quad \langle ABORT\ T_i\rangle$$

Checkpoint:

$$\langle START\ CKPT(T_1, T_2, \ldots)\rangle, \qquad \langle END\ CKPT\rangle$$

5.2 WAL idea

Write-ahead logging requires the relevant log information to reach stable storage before the modified database page is written:

$$\boxed{LOG\ first\ \Rightarrow\ DATA\ page\ later}$$

5.3 Deferred update / NO-UNDO-REDO

Uncommitted changes are not written to the database. Therefore:

$$\boxed{REDO\ winners;\quad no\ UNDO\ losers}$$

If log record stores new value:

$$REDO(\langle T_i, X, new\rangle):\quad X \leftarrow new$$

5.4 Immediate update / UNDO-REDO

Database pages may contain both committed and uncommitted changes at crash time:

$$\boxed{REDO\ committed\ winners;\quad UNDO\ uncommitted\ losers}$$

For:

$$\langle T_i, X, old, new\rangle$$

use:

$$REDO:\ X \leftarrow new,\qquad UNDO:\ X \leftarrow old$$

Typically redo is conceptually forward and undo is performed backward through the log for losers.

5.5 Winner/loser classification

At crash:

$$Winners = \{T_i \mid \langle COMMIT\ T_i\rangle \text{ exists before crash}\}$$

$$Losers = \{T_i \mid T_i \text{ started but did not commit before crash}\}$$

Answer template: "The protocol is _____ because _____. Winners = $\{\cdots\}$; losers = $\{\cdots\}$. REDO: ... . UNDO: ... . Final recovered values: ... ."

6. Functional Dependencies and Normalization (1 point)

6.1 FD definition

For relation schema R, an FD $X \to Y$ holds iff for any legal tuples t_1, t_2:

$$t_1[X] = t_2[X] \Rightarrow t_1[Y] = t_2[Y]$$

6.2 Armstrong axioms

$$\begin{aligned}\text{Reflexivity:}\quad & Y \subseteq X \Rightarrow X \to Y\\ \text{Augmentation:}\quad & X \to Y \Rightarrow XZ \to YZ\\ \text{Transitivity:}\quad & X \to Y \wedge Y \to Z \Rightarrow X \to Z\end{aligned}$$

Useful derived rules:

$$\begin{aligned}\text{Union:}\quad & X \to Y \wedge X \to Z \Rightarrow X \to YZ\\ \text{Decomposition:}\quad & X \to YZ \Rightarrow X \to Y \wedge X \to Z\\ \text{Pseudotransitivity:}\quad & X \to Y \wedge WY \to Z \Rightarrow WX \to Z\end{aligned}$$

6.3 Closure algorithm

Initialize:

$$X^+ \leftarrow X$$

Repeat for every $Y \to Z \in F$:

$$Y \subseteq X^+ \Rightarrow X^+ \leftarrow X^+ \cup Z$$

until no change.

Key criterion:

$$X \text{ superkey} \iff X^+ = R$$

Candidate key criterion:

$$X \text{ candidate key} \iff X^+ = R \ \wedge \ \forall A \in X, (X - \{A\})^+ \neq R$$

6.4 Normal forms

Let $X \to A$ be nontrivial.

2NF: no non-prime attribute is partially dependent on a proper subset of a candidate key.

3NF:

$$\boxed{X \text{ is a superkey } \vee A \text{ is prime}}$$

for every nontrivial FD $X \to A$.

BCNF:

$$\boxed{X \text{ must be a superkey}}$$

for every nontrivial FD $X \to A$.

Thus:

$$BCNF \Rightarrow 3NF \Rightarrow 2NF$$

6.5 Binary lossless-join test

For decomposition $R \to R_1, R_2$:

$$\boxed{(R_1 \cap R_2) \to R_1 \quad \text{or} \quad (R_1 \cap R_2) \to R_2}$$

under F^+ is sufficient and necessary for lossless binary decomposition.

Example:

$$R(A,B,C,D),\ R_1(A,C,D),\ R_2(A,B)$$

If $A \to CD$, then $A \to R_1$, hence lossless.

6.6 BCNF decomposition step

If $X \to Y$ violates BCNF in R:

$$R_1 = X \cup Y$$

$$R_2 = R - (Y - X)$$

Repeat until all components satisfy BCNF.

Exam order matters: compute closures and candidate keys first. You cannot reliably decide 3NF/BCNF before knowing which determinants are superkeys and which attributes are prime.

7. Concurrency Execution and Anomalies (1 point)

7.1 Interleaving principle

A legal interleaving preserves the internal order of each transaction. If:

$$T_1 : o_{11} \prec o_{12} \prec o_{13}, \qquad T_2 : o_{21} \prec o_{22}$$

then any schedule must preserve those two local orders even though operations may interleave globally.

7.2 Lost update

Initial $X = 100$:

$$r_1(X = 100) \quad r_2(X = 100) \quad w_1(X = 90) \quad w_2(X = 120)$$

T_1's update is overwritten. The final 120 reflects only T_2's local computation.

7.3 Dirty read

$$w_1(X) \quad r_2(X) \quad a_1$$

T_2 consumed an uncommitted value that is later rolled back.

7.4 Nonrecoverable pattern

$$w_1(X) \quad r_2(X) \quad c_2 \quad a_1$$

T_2 commits before the transaction whose uncommitted value it read. This is not recoverable.

7.5 Write skew intuition

Two transactions read a shared invariant over different rows, each sees it satisfied, then each writes a different row. No direct write-write conflict may occur on the same item, but the final state violates the cross-row invariant. Serializable isolation prevents the anomalous joint outcome.

8. Query Optimization and I/O Cost (1 point)

8.1 Basic file parameters

Use common textbook notation:

$$\begin{aligned} r &= \text{number of records (tuples)} \\ b &= \text{number of file blocks/pages} \\ bfr &= \text{blocking factor (records per block)} \\ s &= \text{selection cardinality (qualifying records)} \\ x &= \text{number of index levels / index search block accesses} \end{aligned}$$

Approximate pages:

$$\boxed{b = \left\lceil \frac{r}{bfr} \right\rceil}$$

8.2 SELECT cost formulas from the textbook model

Linear scan

$$\boxed{CS_1 = b}$$

(For an equality on a key, average early-stop scan may be about $b/2$, depending on assumptions.)

Binary search on ordered file

$$\boxed{CS_2 \approx \log_2 b + \left\lceil \frac{s}{bfr} \right\rceil - 1}$$

For unique equality ($s = 1$), this reduces approximately to $\log_2 b$ under the textbook convention.

Primary index, single record

$$\boxed{CS_{3a} = x + 1}$$

(one block per index level plus one data block).

Hash key, single record

Typical textbook estimates:

$$\boxed{CS_{3b} \approx 1 \text{ for static/linear hashing,} \qquad \approx 2 \text{ for extendible hashing}}$$

Ordering index for range

Very rough half-file estimate:

$$\boxed{CS_4 \approx x + \frac{b}{2}}$$

Use histogram/selectivity information when available instead of blindly assuming half.

Clustering index, nonkey equality

$$\boxed{CS_5 = x + \left\lceil \frac{s}{bfr} \right\rceil}$$

Because qualifying records are clustered into contiguous file blocks.

Secondary B$^+$-tree, unique equality

$$\boxed{CS_6 = x + 1}$$

Secondary B$^+$-tree, nonkey equality — worst case

If qualifying records are unclustered:

$$\boxed{CS_{6a} = x + 1 + s}$$

The s term reflects potentially one data-block access per qualifying record.

8.3 Selectivity

Selection selectivity:

$$sel = \frac{s}{r}, \qquad 0 \le sel \le 1$$

If uniformly distributed over d distinct values for equality:

$$sel \approx \frac{1}{d}, \qquad s \approx \frac{r}{d}$$

8.4 Join selectivity and cardinality

For join condition c:

$$\boxed{js = \frac{|R \bowtie_c S|}{|R|\,|S|}}$$

Approximate equality-join selectivity:

$$\boxed{js \approx \frac{1}{\max(NDV(A,R), NDV(B,S))}}$$

Then join cardinality:

$$\boxed{jc = |R \bowtie_c S| = js\,|R|\,|S|}$$

8.5 B$^+$-tree vs hashing decision

Predicate	Hash	B$^+$-tree
$A = c$ equality	excellent	excellent
$A < c$, $A > c$, BETWEEN	poor for ordered traversal	excellent
ORDER BY / MIN / MAX	no ordering benefit	ordered access helps
Many matches, unclustered index	can still be expensive	can be very expensive; scan may win

8.6 Core heuristic rules

Push selections and projections down:

$$\sigma \text{ early}, \qquad \pi \text{ early}$$

to reduce intermediate relation sizes before joins.

For conjunctive selections:

$$\sigma_{c_1 \wedge c_2 \wedge \cdots \wedge c_n}(R) \equiv \sigma_{c_1}(\sigma_{c_2}(\cdots \sigma_{c_n}(R) \cdots))$$

Selections commute:

$$\sigma_{c_1}(\sigma_{c_2}(R)) \equiv \sigma_{c_2}(\sigma_{c_1}(R))$$

Join is commutative and associative for ordinary inner joins:

$$R \bowtie S \equiv S \bowtie R$$

$$(R \bowtie S) \bowtie T \equiv R \bowtie (S \bowtie T)$$

which is why join order can be optimized.

9. Fast Decision Trees

9.1 English query → formal method

Phrase	Think immediately
"at least one"	$\exists$, join, EXISTS
"none / never"	$\neg\exists$, anti-join idea, NOT EXISTS, Universe−Bad
"all / every"	$\forall$, $\neg\exists$ counterexample, division $\div$
"at least two distinct"	self-join + renamed copies + $\neq$
"exactly two"	AtLeast2 − AtLeast3 (classical RA), or GROUP BY/HAVING COUNT(DISTINCT...) = 2 in SQL
"highest / maximum"	aggregate/MAX, ranking, or self-join difference trick in classical RA
"same department/faculty"	equality predicate across joined tuples
"no duplicate"	key/UNIQUE or composite key

9.2 Schedule → answer

1. List nodes $T_1, \ldots, T_n$.
2. For each item X, inspect conflicting pairs only.
3. Add all edges $T_i \to T_j$.
4. Cycle? If yes: not conflict-serializable. If no: enumerate topological orders.
5. If recoverability asked: build reads-from dependencies and compare commit positions.
6. If 2PL asked: verify no new lock after first unlock; for strict 2PL, verify write locks held to commit/abort.
7. If deadlock asked: draw wait-for edges and find cycles.

9.3 FD → answer

1. Compute requested closure X^+.
2. Find all minimal superkeys (candidate keys).
3. Mark prime attributes.
4. Test BCNF: every determinant superkey?
5. If no, test 3NF: determinant superkey OR RHS prime?
6. If decomposition given: test lossless join using intersection closure.

10. One-Page Ultra-Condensed Sheet

RA: $\sigma_\theta(R)$ select; $\pi_A(R)$ project; ρ rename; $\times$ product; $\bowtie$ join; $\div$ all/every; $R - S$ difference.
At least $2 =$ self-join with distinct values. Exactly $2 = \geq 2 - \geq 3$.
Division: $T_1 = \pi_Y(R)$, $T_2 = \pi_Y((S \times T_1) - R)$, $T = T_1 - T_2$.

TRC: $\{t \mid P(t)\}$; $\exists$ at least one; $\neg\exists$ none; $\forall x P(x) \equiv \neg\exists x \neg P(x)$.

Constraints: Row-local $\rightarrow$ CHECK; existence $\rightarrow$ FK; uniqueness $\rightarrow$ PK/UNIQUE; cross-table $\rightarrow$ assertion/trigger. Formal: $\neg\exists$(violating tuples).

Conflicts: Same item + different Tx + at least one write. Edge from earlier op's Tx to later op's Tx. Acyclic precedence graph $\Leftrightarrow$ CSR. Serial orders = topological sorts.

Recovery properties: Recoverable: writer commit before reader commit. Cascadeless: writer commit before reader read. Strict: no read/write of uncommitted written item. Strict $\Rightarrow$ cascadeless $\Rightarrow$ recoverable.

2PL: Grow=lock, shrink=unlock; no new lock after first unlock. Strict 2PL holds X-locks until commit/abort. Wait-for cycle = deadlock.

Recovery: Deferred/NO-UNDO-REDO: REDO winners, ignore losers. Immediate/UNDO-REDO: REDO winners, UNDO losers. $\langle T, X, old, new\rangle$: redo new, undo old.

FD: $X \rightarrow Y$; closure X^+. Superkey iff $X^+ = R$. Candidate key=minimal superkey. 3NF: determinant superkey OR RHS prime. BCNF: determinant superkey. Binary lossless: $(R_1 \cap R_2) \rightarrow R_1$ or R_2.

Costs: $b = \lceil r/bfr \rceil$; scan $CS_1 = b$; primary/B$^+$ key $x+1$; hash equality ≈ 1 static/linear or 2 extendible; clustering $x + \lceil s/bfr \rceil$; secondary nonkey worst-case $x + 1 + s$.
$js = |R \bowtie S|/(|R||S|)$; $jc = js|R||S|$.

Reference scope

This cheatsheet paraphrases and reorganizes notation and concepts from *Fundamentals of Database Systems*, 7th ed., for exam revision. Particularly relevant book sections are: ER modeling (Ch. 3), relational constraints and SQL (Ch. 5–7), relational algebra/calculus and ER mapping (Ch. 8–9), FDs/normalization (Ch. 14–15), indexing and query optimization (Ch. 17–19), transactions/concurrency/recovery (Ch. 20–22). It is not a replacement for the textbook and intentionally does not reproduce long passages verbatim.